



Voz Autista

Dispositivo de Comunicacao Assistiva para Pessoas com Autismo

Edicao Maker — Documento Tecnico Completo

Projeto de Prototipacao de Hardware

Plataforma: Arduino Uno R3

Versao do Documento: 1.0 — Abril 2026

Conteudo: Visao Geral | Lista de Materiais | Circuito Corrigido
Codigo-Fonte | Case para Impressao 3D | Montagem | Testes

Sumario

1. Visao Geral do Projeto

1.1 Objetivo

1.2 Publico-Alvo

1.3 Funcionalidades

1.4 Arquitetura do Sistema

2. Lista de Materiais (BOM)

2.1 Componentes Eletronicos

2.2 Materiais para o Case

2.3 Ferramentas Necessarias

3. Correcoes no Circuito

3.1 Problemas Identificados

3.2 Solucoes Aplicadas

4. Esquema Eletrico (Diagrama)

4.1 Diagrama de Ligacao

4.2 Pinagem Completa

5.Codigo-Fonte (Arduino)

5.1 Bibliotecas Necessarias

5.2 Codigo Completo Comentado

5.3 Estrutura de Dados

6. Preparacao do Cartao SD

7. Case para Impressao 3D

7.1 Dimensoes e Especificacoes

7.2 Parametros do OpenSCAD

7.3 Configuracoes de Impressao

8. Montagem Passo a Passo

9. Testes e Validacao

10. Solucao de Problemas

11. Arquivos do Projeto

1. Visao Geral do Projeto

1.1 Objetivo

O **Voz Autista** e um dispositivo de comunicacao assistiva projetado para pessoas com Transtorno do Espectro Autista (TEA) que apresentam dificuldades na comunicacao verbal. O dispositivo permite que o usuario navegue por categorias de palavras e frases pre-definidas e reproduza o audio correspondente atraves de um alto-falante.

1.2 Publico-Alvo

- Crianças e adultos com TEA que possuem dificuldades de comunicacao verbal
- Cuidadores, terapeutas e familiares que buscam ferramentas de CAA (Comunicacao Aumentativa e Alternativa)
- Profissionais de saude e educacao especial

1.3 Funcionalidades

Funcionalidade	Descricao
Navegacao por Categorias	3 categorias: Necessidades, Sentimentos, Acoes (5 palavras cada)
Reproducao de Audio	MP3 pre-gravados via DFPlayer Mini com alto-falante
Display LCD	Tela 16x2 mostrando categoria e palavra selecionada
Interface Simples	5 botoes coloridos com navegacao circular
Feedback Visual	LCD mostra "Falando..." durante reproducao
Deteccao de Erro	Aviso no LCD se DFPlayer nao for detectado

1.4 Arquitetura do Sistema

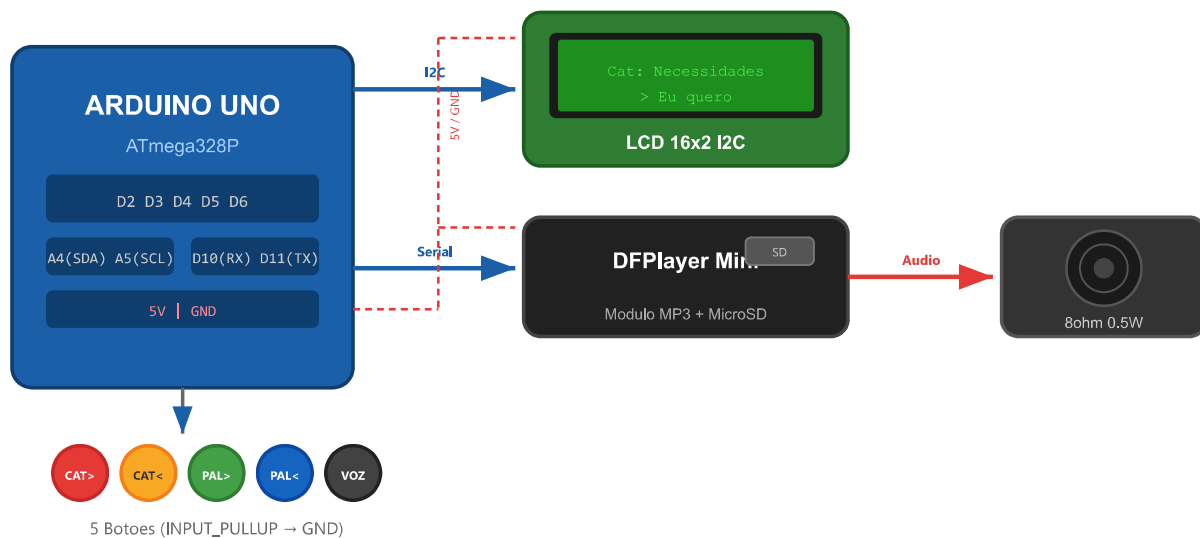


Figura 1 — Diagrama de blocos da arquitetura do sistema Voz Autista

Nota sobre Acessibilidade:

Os botoes sao coloridos (vermelho, amarelo, verde, azul, preto) para facilitar a identificacao visual. A navegacao e circular — ao chegar no final da lista, volta para o inicio automaticamente. O debounce de 200ms previne acionamentos acidentais.

2. Lista de Materiais (BOM)

2.1 Componentes Eletronicos

#	Componente	Qtd	Especificacao	Observacao
1	Arduino Uno R3	1	ATmega328P, 5V	Clone compativel aceito
2	LCD 16x2 com modulo I2C	1	Endereco 0x27	Verificar endereco com scanner I2C
3	DFPlayer Mini MP3	1	Suporta FAT32 microSD	Com slot para cartao
4	Cartao microSD	1	FAT32, ate 32GB	Classe 4 ou superior
5	Alto-falante	1	8 ohm, 0.5W, 40mm	Compativel com DFPlayer
6	Botao push momentaneo	5	12mm diametro	Cores: vermelho, amarelo, verde, azul, preto
7	Resistor 1k ohm	1	1/4W, 5%	Protecao RX do DFPlayer
8	Cabos jumper	~20	Macho-macho	Diversas cores recomendado
9	Protoboard 830 pontos	1	Standard	Opcional se soldar direto

Economia no circuito corrigido:

Os 5 resistores pull-down de 10k ohm do projeto original foram **removidos**. O codigo utiliza `INPUT_PULLUP` do Arduino (pull-up interno de ~47k ohm), tornando os resistores externos desnecessarios.

2.2 Materiais para o Case (Impressao 3D)

#	Material	Qtd	Observacao
1	Filamento PLA ou PETG	~80g	Cor a escolha (azul claro recomendado)
2	Parafusos M3 x 8mm	4	Fixacao tampa-base
3	Parafusos M2.5 x 6mm	4	Fixacao do Arduino na base
4	Insertos de calor M3	4	Opcional (melhora durabilidade)

2.3 Ferramentas Necessarias

- Chave de fenda Phillips pequena
- Ferro de solda e estanho (se não usar protoboard)
- Impressora 3D FDM (ou acesso a serviço de impressão)
- Computador com Arduino IDE instalado
- Cabo USB tipo B (para programar o Arduino)
- Multímetro (opcional, para debug)

3. Correcoes no Circuito

Durante a analise do projeto original, foram identificados **3 problemas criticos** que impediriam o funcionamento correto do dispositivo. Todas as correcoes ja foram aplicadas nos arquivos deste documento.

3.1 Problemas Identificados

Problema 1: Conflito Pull-Up vs Pull-Down

CRITICO — Circuito nao funcionaria

O codigo original usava `INPUT_PULLUP` (puxa o pino para HIGH internamente) e verificava `digitalRead() == LOW` para detectar pressionamento. Porem, o esquema eletrico mostrava resistores pull-DOWN (10k ohm para GND) com botoes conectados ao 5V. Nessa configuracao, o botao pressionado geraria HIGH, nao LOW — resultado: os botoes nunca seriam detectados corretamente.

Problema 2: DFPlayer Completamente Desativado

FUNCIONAL — Dispositivo mudo

Todo o codigo relacionado ao DFPlayer Mini estava comentado. O dispositivo exibia texto no LCD mas nunca reproduzia audio — derrotando o proposito principal do projeto.

Problema 3: Tipo Incorreto para millis()

LATENTE — Falha apos ~25 dias

A variavel `lastButtonPressTime` usava tipo `long` (com sinal). A funcao `millis()` retorna `unsigned long`. Apos ~24.8 dias de uso continuo, o valor estouraria para negativo, causando comportamento erratico nos botoes.

Comparacao Visual: Antes vs Depois

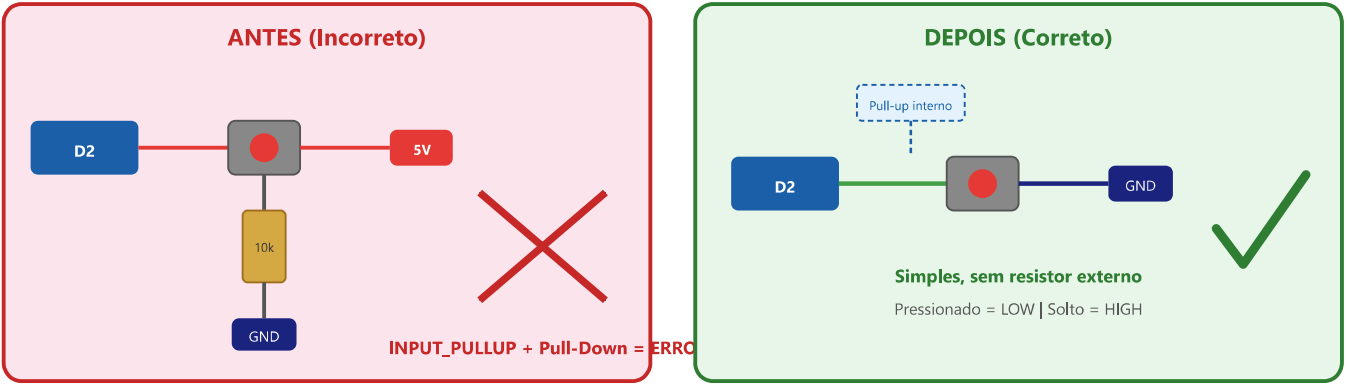


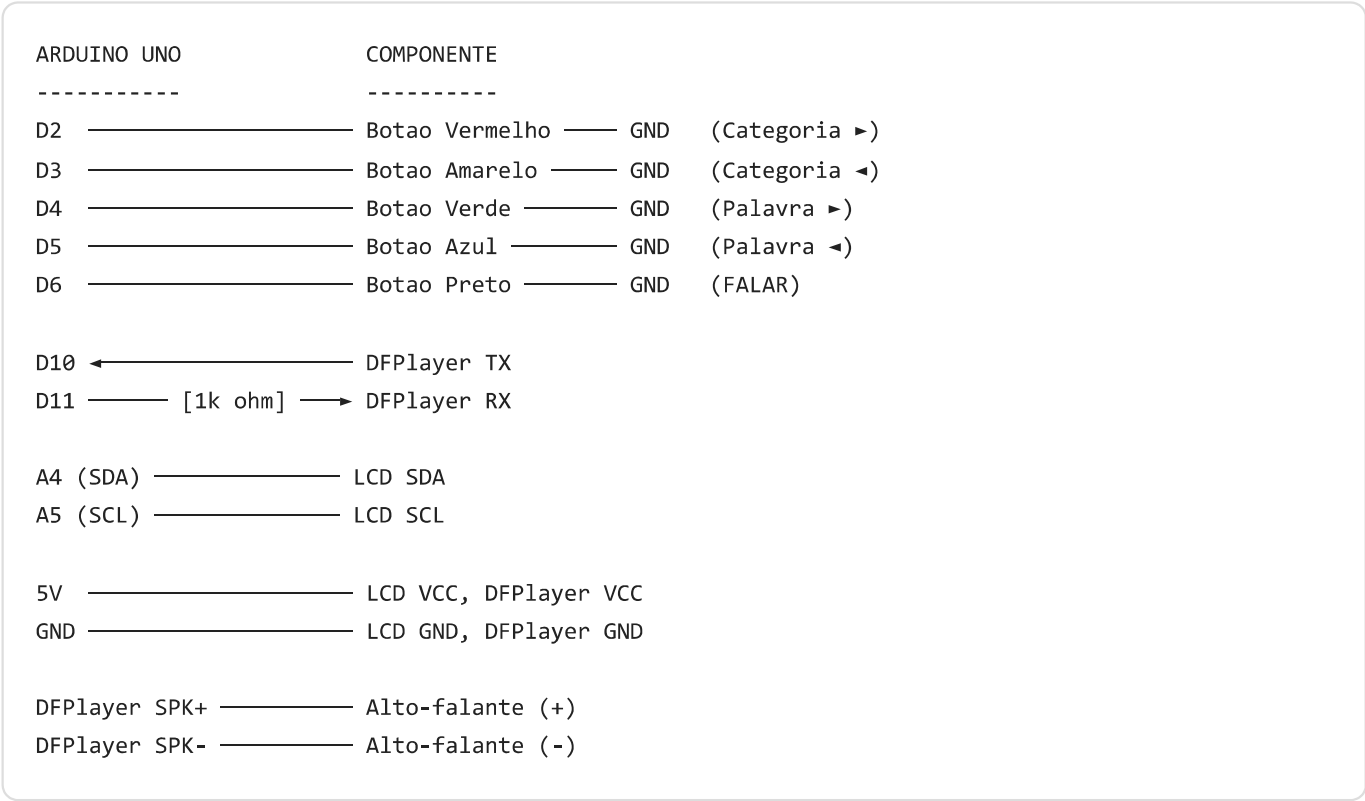
Figura 2 — Comparacao da ligacao dos botoes: projeto original (erro) vs corrigido

3.2 Solucoes Aplicadas

Problema	Solucao	Impacto
Pull-Up vs Pull-Down	Removidos os 5 resistores 10k ohm. Botoes agora conectam pino → GND diretamente. O INPUT_PULLUP interno do Arduino e suficiente.	Economia de 5 componentes, circuito mais simples
DFPlayer desativado	Codigo ativado com biblioteca DFRobotDFPlayerMini . Adicionada deteccao de erro e feedback visual no LCD.	Dispositivo agora reproduz audio
Tipo de millis()	Alterado de long para unsigned long .	Previne bug apos uso prolongado

4. Esquema Eletrico

4.1 Diagrama de Ligacao Simplificado



4.2 Diagrama Visual Completo do Circuito

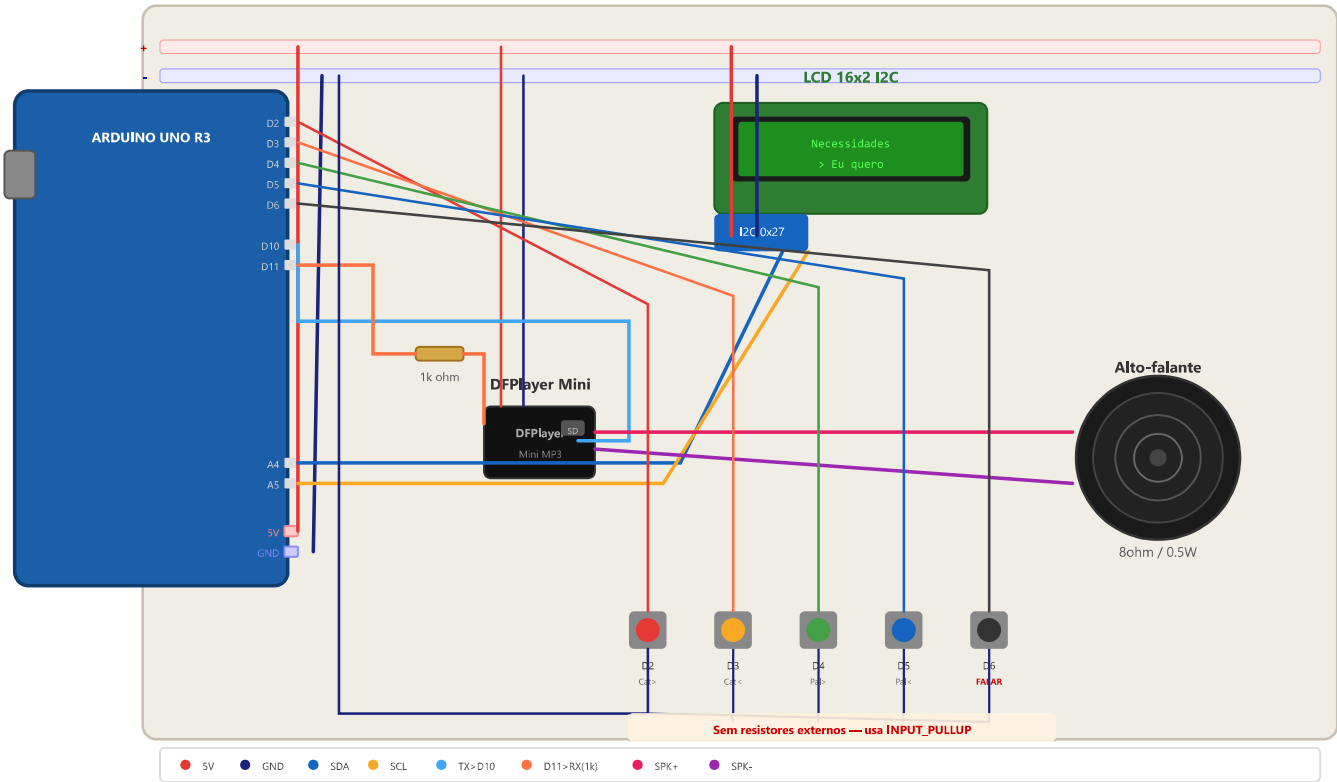


Figura 3 — Diagrama completo de ligacao do circuito corrigido (estilo Fritzing)

4.2 Pinagem Completa do Arduino Uno

Pino	Tipo	Funcao	Conectado a
D2	Digital (INPUT_PULLUP)	Botao Categoria ►	Botao Vermelho → GND
D3	Digital (INPUT_PULLUP)	Botao Categoria ◀	Botao Amarelo → GND
D4	Digital (INPUT_PULLUP)	Botao Palavra ►	Botao Verde → GND
D5	Digital (INPUT_PULLUP)	Botao Palavra ◀	Botao Azul → GND
D6	Digital (INPUT_PULLUP)	Botao FALAR	Botao Preto → GND
D10	SoftwareSerial RX	Recebe dados do DFPlayer	DFPlayer TX (direto)
D11	SoftwareSerial TX	Envia dados ao DFPlayer	DFPlayer RX (via 1k ohm)
A4	I2C SDA	Dados do LCD	LCD modulo I2C SDA
A5	I2C SCL	Clock do LCD	LCD modulo I2C SCL
5V	Alimentacao	+5V	LCD VCC, DFPlayer VCC
GND	Alimentacao	Terra	LCD GND, DFPlayer GND, todos os botoes

Importante — Resistor de 1k ohm:

O resistor de 1k ohm entre D11 (TX do Arduino) e RX do DFPlayer e **obrigatorio**

. Ele protege o pino RX do DFPlayer que opera em 3.3V contra os 5V do Arduino. Sem ele, o DFPlayer pode ser danificado.

5. Código-Fonte (Arduino)

5.1 Bibliotecas Necessarias

Instalar pelo Arduino IDE: **Sketch** → **Incluir Biblioteca** → **Gerenciar Bibliotecas**

Biblioteca	Autor	Versao Min.	Funcao
LiquidCrystal I2C	Frank de Brabander	1.1.2	Controle do LCD via I2C
DFRobotDFPlayerMini	DFRobot	1.0.5	Controle do modulo DFPlayer Mini
SoftwareSerial	(inclusa no Arduino IDE)	-	Comunicacao serial com DFPlayer

5.2 Código Completo Comentado

Arquivo: voz_autista_maker_code.ino

```
// =====
// Voz Autista - Edicao Maker (Codigo Corrigido)
// Dispositivo de Comunicacao Assistiva para pessoas com autismo
// =====

#include <LiquidCrystal_I2C.h>
#include <SoftwareSerial.h>
#include <DFRobotDFPlayerMini.h>

// --- Configuracao do LCD ---
LiquidCrystal_I2C lcd(0x27, 16, 2);

// --- Configuracao do DFPlayer Mini ---
SoftwareSerial mySerial(10, 11); // RX=D10, TX=D11
DFRobotDFPlayerMini dfPlayer;

// --- Pinos dos botoes (conectar entre pino e GND) ---
const int btnCategoryUp = 2; // Vermelho - Categoria ►
const int btnCategoryDown = 3; // Amarelo - Categoria ◄
const int btnWordUp = 4; // Verde - Palavra ►
const int btnWordDown = 5; // Azul - Palavra ◄
const int btnSpeak = 6; // Preto - FALAR

// --- Estrutura de dados ---
struct Category {
    const char* name;
    const char* words[5];
    int audioId[5];
    int numWords;
};

// --- Categorias e palavras ---
Category categories[] = {
    {"Necessidades", {"Eu quero", "Comer", "Beber", "Banheiro", "Dormir"}, {1, 2, 3, 4, 5}, 5},
    {"Sentimentos", {"Feliz", "Triste", "Bravo", "Calmo", "Assustado"}, {6, 7, 8, 9, 10}, 5},
    {"Acoes", {"Brincar", "Ajuda", "Sair", "Entrar", "Ouvir"}, {11, 12, 13, 14, 15}, 5}
};

const int NUM_CATEGORIES = sizeof(categories) / sizeof(categories[0]);
int currentCategoryIndex = 0;
int currentWordIndex = 0;
int volume = 20;

// --- Debounce ---
unsigned long lastButtonPressTime = 0;
const unsigned long debounceDelay = 200;

bool dfPlayerReady = false;

void setup() {
    Serial.begin(9600);
    mySerial.begin(9600);

    lcd.init();
    lcd.backlight();
    lcd.print("Voz Autista");
    lcd.setCursor(0, 1);
    lcd.print("Edicao Maker");

    pinMode(btnCategoryUp, INPUT_PULLUP);
    pinMode(btnCategoryDown, INPUT_PULLUP);
    pinMode(btnWordUp, INPUT_PULLUP);
    pinMode(btnWordDown, INPUT_PULLUP);
    pinMode(btnSpeak, INPUT_PULLUP);
}
```

```
delay(1000);
if (dfPlayer.begin(mySerial)) {
    dfPlayerReady = true;
    dfPlayer.volume(volume);
} else {
    lcd.clear();
    lcd.print("ERRO: DFPlayer");
    lcd.setCursor(0, 1);
    lcd.print("Cheque conexoes");
    delay(2000);
}

lcd.clear();
displayCurrentSelection();
}

void loop() {
    unsigned long currentTime = millis();
    if ((currentTime - lastButtonPressTime) < debounceDelay) return;

    if (digitalRead(btnCategoryUp) == LOW) {
        currentCategoryIndex = (currentCategoryIndex + 1) % NUM_CATEGORIES;
        currentWordIndex = 0;
        displayCurrentSelection();
        lastButtonPressTime = currentTime;
    }
    else if (digitalRead(btnCategoryDown) == LOW) {
        currentCategoryIndex = (currentCategoryIndex - 1 + NUM_CATEGORIES) % NUM_CATEGORIES;
        currentWordIndex = 0;
        displayCurrentSelection();
        lastButtonPressTime = currentTime;
    }
    else if (digitalRead(btnWordUp) == LOW) {
        int n = categories[currentCategoryIndex].numWords;
        currentWordIndex = (currentWordIndex + 1) % n;
        displayCurrentSelection();
        lastButtonPressTime = currentTime;
    }
    else if (digitalRead(btnWordDown) == LOW) {
        int n = categories[currentCategoryIndex].numWords;
        currentWordIndex = (currentWordIndex - 1 + n) % n;
        displayCurrentSelection();
        lastButtonPressTime = currentTime;
    }
    else if (digitalRead(btnSpeak) == LOW) {
        speakCurrentWord();
        lastButtonPressTime = currentTime;
    }
}

void displayCurrentSelection() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Cat: ");
    lcd.print(categories[currentCategoryIndex].name);
    lcd.setCursor(0, 1);
    lcd.print("> ");
    lcd.print(categories[currentCategoryIndex].words[currentWordIndex]);
}

void speakCurrentWord() {
    int id = categories[currentCategoryIndex].audioId[currentWordIndex];
    const char* word = categories[currentCategoryIndex].words[currentWordIndex];
```

```
lcd.clear();  
lcd.print("Falando...");  
lcd.setCursor(0, 1);  
lcd.print(word);  
  
if (dfPlayerReady) {  
    dfPlayer.playFolder(1, id); // Pasta 01, arquivo id  
}  
  
delay(500);  
displayCurrentSelection();  
}
```

5.3 Estrutura de Dados — Categorias e Palavras

Categoria	Palavras (5)	IDs Audio
Necessidades	Eu quero, Comer, Beber, Banheiro, Dormir	1 - 5
Sentimentos	Feliz, Triste, Bravo, Calmo, Assustado	6 - 10
Acoes	Brincar, Ajuda, Sair, Entrar, Ouvir	11 - 15

6. Preparacao do Cartao SD

Formatacao

1. Inserir o microSD no computador
2. Formatar como **FAT32** (nao exFAT, nao NTFS)
3. Tamanho de alocao: padrao

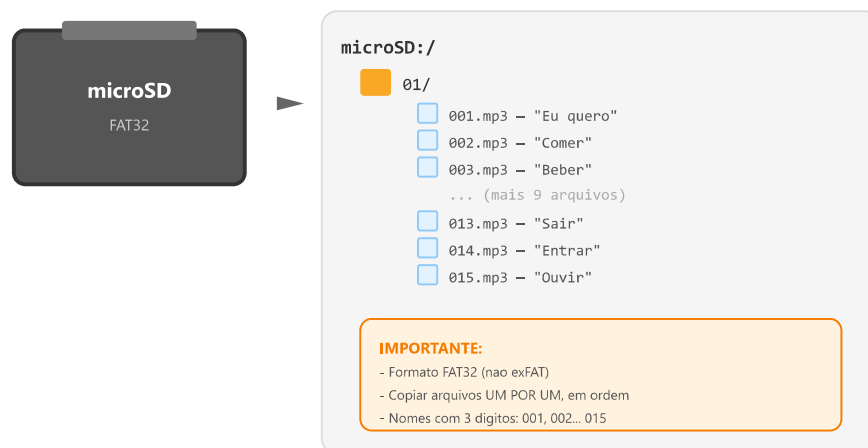


Figura 7 — Estrutura do cartao microSD para o DFPlayer Mini

Estrutura de Pastas e Arquivos

Os arquivos MP3 devem estar na pasta 01 na raiz do cartao, com numeracao sequencial de 3 digitos:

Arquivo	Conteudo do Audio	Categoria
/01/001.mp3	"Eu quero"	Necessidades
/01/002.mp3	"Comer"	Necessidades
/01/003.mp3	"Beber"	Necessidades
/01/004.mp3	"Banheiro"	Necessidades
/01/005.mp3	"Dormir"	Necessidades
/01/006.mp3	"Feliz"	Sentimentos
/01/007.mp3	"Triste"	Sentimentos
/01/008.mp3	"Bravo"	Sentimentos
/01/009.mp3	"Calmo"	Sentimentos
/01/010.mp3	"Assustado"	Sentimentos
/01/011.mp3	"Brincar"	Acoes
/01/012.mp3	"Ajuda"	Acoes
/01/013.mp3	"Sair"	Acoes
/01/014.mp3	"Entrar"	Acoes
/01/015.mp3	"Ouvir"	Acoes

Como gerar os audios:

Utilize um servico de Text-to-Speech (TTS) em portugues brasileiro para gerar os arquivos MP3. Voz clara e com velocidade reduzida e ideal para o publico-alvo. Recomendacoes: Google Cloud TTS, Amazon Polly, ou ferramentas online gratuitas de TTS.

Atencao:

O DFPlayer Mini e sensivel a ordem de copia dos arquivos no SD. Copie os arquivos **um por um, em ordem numerica**

(001 primeiro, depois 002, etc.). Nao copie todos de uma vez — alguns cartoes reordenam os arquivos internamente.

7. Case para Impressao 3D

7.1 Dimensoes e Especificacoes

Dimensao	Valor
Dimensao externa	175 x 115 x 47 mm
Dimensao interna	170 x 110 x 42 mm
Espessura da parede	2.5 mm
Raio dos cantos	8 mm (arredondados — seguro para crianas)
Altura da base	30 mm (contem todos os componentes)
Altura da tampa	12 mm (LCD + botoes + speaker)
Peso estimado (PLA)	70 - 80g

Vista Superior da Tampa (Layout)

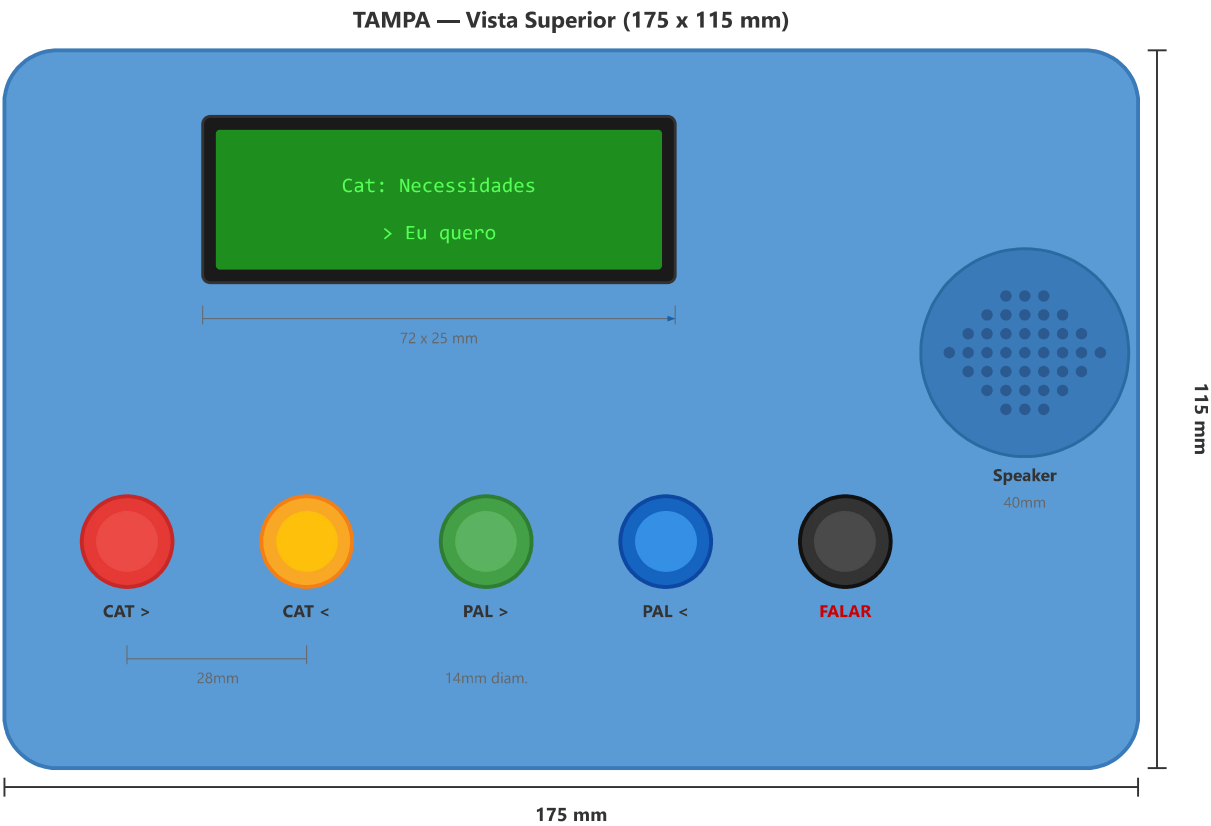


Figura 4 — Vista superior da tampa: posicao do LCD, 5 botoes coloridos e grade do alto-falante

Vista Lateral (Corte)

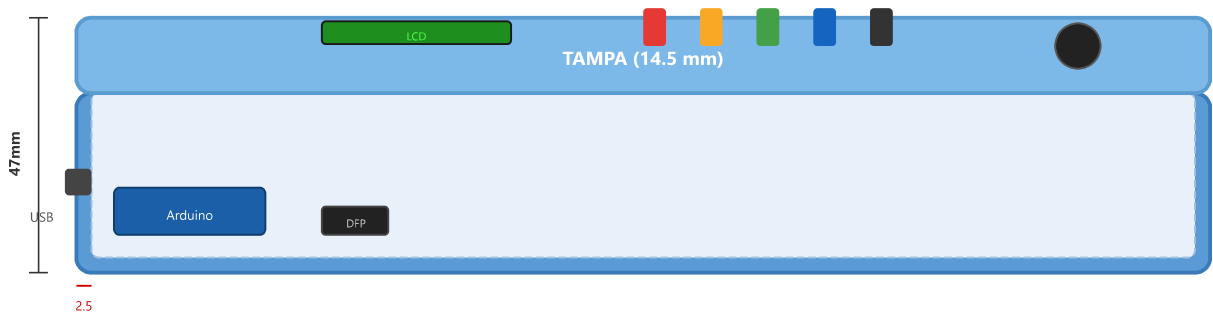


Figura 5 — Vista lateral em corte: distribuicao dos componentes dentro do case

Aberturas e Recortes

Elemento	Dimensao	Posicao
Janela LCD	72 x 25 mm	Tampa (topo, centro-esquerda)
Furos botoes (x5)	14 mm diametro, espacados 28 mm	Tampa (borda inferior)
Grade alto-falante	40 mm diametro (furos de 2.5 mm)	Tampa (canto direito)
Abertura USB	14 x 12 mm	Base (lateral esquerda)
Abertura Jack	12 x 11 mm	Base (lateral esquerda)
Abertura SD Card	14 x 4 mm	Base (lateral direita)

7.2 Parametros do OpenSCAD

Arquivo: case_3d/case_voz_autista.scad — Todos os valores sao ajustaveis:

```
// Dimensoes internas
case_width  = 170; // Largura
case_depth  = 110; // Profundidade
case_height = 42;  // Altura total
wall        = 2.5; // Espessura da parede
corner_r    = 8;   // Raio dos cantos
tolerance   = 0.3; // Folga de encaixe

// Botoes
btn_diameter = 14; // Diametro do furo
btn_spacing  = 28; // Espacamento entre centros

// Alto-falante
spk_diameter = 40; // Diametro da grade
```

7.3 Configuracoes de Impressao Recomendadas

Parametro	Valor Recomendado
Material	PLA ou PETG
Altura de camada	0.2 mm
Preenchimento (infill)	20 - 25%
Perímetros (paredes)	3
Suportes	Nao necessarios
Brim	Recomendado (5 mm)
Temperatura (PLA)	200 - 210 C (extrusora) / 60 C (mesa)
Temperatura (PETG)	230 - 240 C (extrusora) / 80 C (mesa)
Velocidade	50 - 60 mm/s
Tempo estimado — Base	3 - 4 horas
Tempo estimado — Tampa	2 - 3 horas

Como Exportar para STL

1. Instalar o **OpenSCAD** (openscad.org) — ou usar o visualizador HTML incluso (case_3d/visualizar_case_3d.html)
2. Abrir case_3d/exportar_base.scad
3. Pressionar **F6** (Render)
4. Menu **File** → **Export as STL**
5. Repetir com case_3d/exportar_tampa.scad

8. Montagem Passo a Passo



Fluxo de montagem: 5 etapas sequenciais

Figura 6 — Fluxo de montagem em 5 etapas

Etapa 1: Preparacao

1. Imprimir as duas pecas do case (base e tampa)
2. Verificar se todos os furos estao com a folga correta — lixar se necessario
3. Opcionalmente, instalar insertos de calor M3 nos 4 pilares da base (uso de ferro de solda)
4. Separar e identificar todos os componentes eletronicos

Etapa 2: Montagem Eletronica

1. **Fixar o Arduino Uno** na base com 4 parafusos M2.5 x 6mm nos pilares de montagem
2. **Conectar o LCD 16x2 I2C:**
 - VCC → 5V do Arduino
 - GND → GND do Arduino
 - SDA → Pino A4
 - SCL → Pino A5
3. **Conectar o DFPlayer Mini:**
 - VCC → 5V do Arduino
 - GND → GND do Arduino
 - TX (DFPlayer) → Pino D10 (direto)
 - RX (DFPlayer) → Resistor 1k ohm → Pino D11
4. **Conectar o alto-falante:**
 - SPK+ do DFPlayer → Terminal + do alto-falante
 - SPK- do DFPlayer → Terminal - do alto-falante
5. **Conectar os 5 botoes** (cada um entre o pino digital e GND):
 - Vermelho → D2 e GND
 - Amarelo → D3 e GND
 - Verde → D4 e GND
 - Azul → D5 e GND

- o Preto → D6 e GND

6. Inserir o **microSD preparado** no slot do DFPlayer

Etapa 3: Software

1. Instalar o **Arduino IDE** (arduino.cc)
2. Instalar as bibliotecas: LiquidCrystal I2C e DFRobotDFPlayerMini
3. Abrir o arquivo voz_autista_maker_code.ino
4. Selecionar placa "**Arduino Uno**" e a porta COM correta
5. Fazer upload do código (**Ctrl+U**)

Etapa 4: Teste Funcional

1. Após o upload, o LCD deve mostrar "**Voz Autista / Edicao Maker**" por 2 segundos
2. Se o DFPlayer não for detectado, aparecerá "**ERRO: DFPlayer / Cheque conexoes**"
3. Testar navegação: botões vermelho/amarelo mudam categoria, verde/azul mudam palavra
4. Pressionar o botão preto (FALAR) — o áudio deve ser reproduzido pelo alto-falante
5. Verificar no Monitor Serial (9600 baud) as mensagens de debug

Etapa 5: Fechamento do Case

1. Encaixar o LCD na janela retangular da tampa
2. Passar os botões pelos furos circulares da tampa
3. Posicionar o alto-falante alinhado com a grade
4. Fechar a tampa sobre a base (a borda de encaixe guia o alinhamento)
5. Fixar com 4 parafusos M3 x 8mm nos pilares dos cantos

9. Testes e Validacao

Checklist de Testes

#	Teste	Resultado Esperado	Status
1	Ligar o Arduino via USB	LED verde aceso, LCD liga	
2	Tela inicial	"Voz Autista / Edicao Maker" por 2s	
3	DFPlayer detectado	LCD mostra selecao, nao mostra erro	
4	Botao vermelho (Cat ►)	Muda para proxima categoria	
5	Botao amarelo (Cat ◄)	Volta para categoria anterior	
6	Botao verde (Pal ►)	Proxima palavra da categoria	
7	Botao azul (Pal ◄)	Palavra anterior da categoria	
8	Botao preto (FALAR)	LCD mostra "Falando...", audio toca	
9	Navegacao circular	Ultima categoria → primeira (e vice-versa)	
10	Todos os 15 audios	Cada audio corresponde a palavra correta	
11	Volume adequado	Audio claro a 1m de distancia	
12	Debounce	Um clique = uma acao (sem repeticao)	
13	Case fechado	Todos os botoes acessiveis, LCD visivel, som audivel	

10. Solucao de Problemas

Problema	Causa Provavel	Solucao
LCD nao liga	Endereco I2C errado	Rodar sketch <i>I2C Scanner</i> no Arduino. Trocar <code>0x27</code> por <code>0x3F</code> se necessario.
LCD liga mas sem texto	Contraste mal ajustado	Girar o potenciometro azul no modulo I2C atras do LCD com chave Phillips.
DFPlayer nao detectado	Conexao TX/RX invertida	Verificar: TX do DFPlayer vai para D10, RX do DFPlayer vai para D11 (via 1k ohm).
DFPlayer detectado mas sem som	SD card com problema	1) Verificar formato FAT32. 2) Verificar nomes dos arquivos (001.mp3 na pasta 01). 3) Copiar arquivos um por um em ordem.
Audio com ruído/chiado	Interferencia na alimentacao	Adicionar capacitor eletrolitico de 100uF entre VCC e GND do DFPlayer (perna longa no +).
Botao nao responde	Fiacao invertida	Botao deve conectar pino digital ↔ GND (nao pino ↔ 5V). Testar continuidade com multmetro.
Multiplos cliques por pressao	Debounce insuficiente	Aumentar <code>debounceDelay</code> de 200 para 300 ou 400 no codigo.
Erro de compilacao	Biblioteca faltando	Instalar <code>LiquidCrystal I2C</code> e <code>DFRobotDFPlayerMini</code> pelo Gerenciador de Bibliotecas.
Upload falha	Porta COM errada	Verificar em Ferramentas → Porta. Tentar outro cabo USB. Fechar Monitor Serial se estiver aberto.
Case nao fecha	Tolerancia de impressao	Lixar levemente a borda de encaixe. Aumentar <code>tolerance</code> no OpenSCAD para 0.4 ou 0.5.

Scanner I2C (codigo auxiliar)

Se o LCD nao for detectado, carregue este sketch para descobrir o endereco correto:

```
#include <Wire.h>

void setup() {
  Wire.begin();
  Serial.begin(9600);
  Serial.println("Scanning I2C...");
  for (byte addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);
    if (Wire.endTransmission() == 0) {
      Serial.print("Dispositivo em 0x");
      Serial.println(addr, HEX);
    }
  }
  Serial.println("Scan completo.");
}

void loop() {}
```

11. Arquivos do Projeto

Arquivo	Tipo	Descricao
voz_autista_maker_code.ino	Arduino	Codigo-fonte corrigido e completo
voz_autista_diagrama_completo.html	HTML/SVG	Diagrama de circuito interativo estilo Fritzing
GUIA_IMPLEMENTACAO.md	Markdown	Guia de implementacao resumido
DOCUMENTO_COMPLETO_VOZ_AUTISTA.html	HTML	Este documento (salvar como PDF)
case_3d/case_voz_autista.scad	OpenSCAD	Modelo 3D parametrizado do case (base + tampa)
case_3d/exportar_base.scad	OpenSCAD	Auxiliar para exportar base em STL
case_3d/exportar_tampa.scad	OpenSCAD	Auxiliar para exportar tampa em STL
case_3d/visualizar_case_3d.html	HTML/Three.js	Visualizador 3D interativo no navegador
manual_voz_autista_maker.pdf	PDF	Manual maker original
guia_montagem_passo_a_passo_arduino.pdf	PDF	Guia de montagem original
guia_visual_realista_voz_autista.pdf	PDF	Guia visual original